

Modelagem Geométrica

Representações Celulares Adaptativas e Esferas Geodésicas

Relatório de Implementação

2 de Fevereiro de 2026

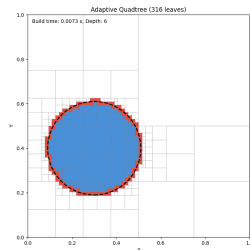
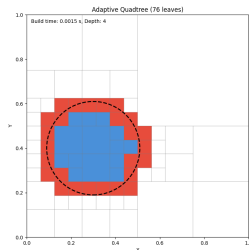
Representação Celular Adaptativa (Quadtree)

Princípio: Subdividir apenas células que intersectam a fronteira

Classificação de Células:

- $\sigma = 0$: Exterior (branco)
- $\sigma = 1$: Interior (azul)
- $\sigma = 2$: Interseção → **subdivide**

Complexidade: $O(L \cdot 2^d)$ células na fronteira



Disco: profundidade 4 (esq.) e 6 (dir.)

Algoritmo

```
if classify(cell) == INTERSECT and depth < max_depth:    subdivide(cell)
```

Esfera Geodésica por Refinamento

Entrada: Icosaedro (12 vértices, 20 faces)

Algoritmo:

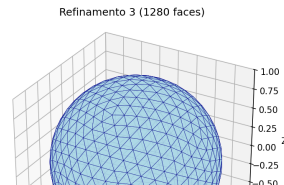
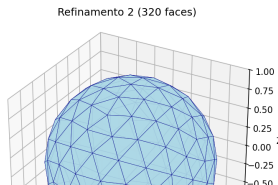
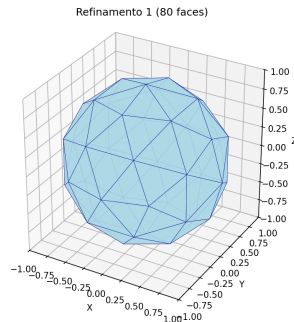
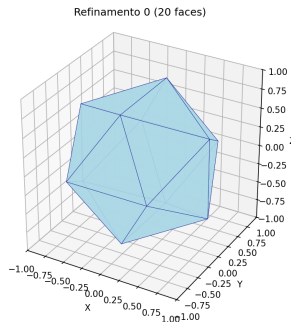
- 1 Calcular pontos médios:

$$m_{ij} = \frac{v_i + v_j}{2}$$

- 2 Projetar na esfera: $m'_{ij} = \frac{m_{ij}}{\|m_{ij}\|}$

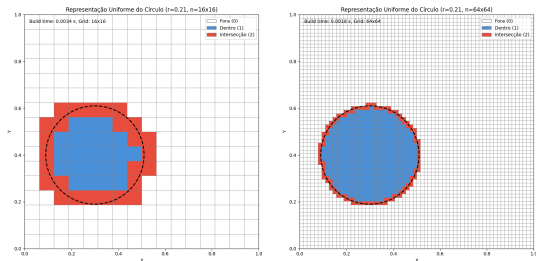
- 3 Criar 4 novos triângulos

Crescimento: $F_k = 4^k \cdot F_0$



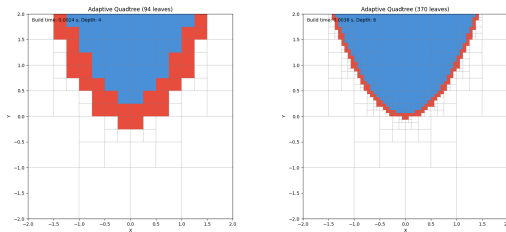
Resultados: Representação Uniforme vs Adaptativa

Uniforme (PyTorch + vmap):



Grade 16×16 (esq.) e 64×64 (dir.)

Adaptativa (Quadtree):



Parábola: profundidade 4 (esq.) e 6 (dir.)

Comparação

Uniforme: Todas as células do mesmo tamanho, vetorizável em GPU. eficiente em memória, foco na fronteira.

Adaptativa: Mais

O que foi implementado:

- Representação uniforme com PyTorch/vmap (GPU)
- Quadtree adaptativa com animações GIF
- Esfera geodésica por refinamento de icosaedro
- Exportação para formato OBJ

Dificuldades:

- 1 Adaptar para vmap (sem `.item()`)
- 2 Unificação de vértices duplicados (hash)
- 3 Classificação de células para parábolas

Arquivos Gerados:

- 72+ imagens PNG
- Animações GIF do processo
- `refined_sphere.obj`

Formas Testadas:

- Disco: $(0.3, 0.4)$, $r = 0.21$
- Parábola: $y \geq x^2 + c$
- Icosaedro $\rightarrow$ Esfera